TPO Ajedrez — Cómo está hecho

IA · Piezas 3D · Base de datos · Arquitectura hexagonal (puertos, adaptadores y núcleo)

1. ¿Cómo implementamos la IA?

La IA está en **core/ai**, dentro del núcleo, y no usa ninguna librería externa ni machine learning. Es una IA **heurística de una jugada** (mira un movimiento hacia adelante): prueba cada jugada legal, le pone un puntaje y elige la mejor. Se arma combinando dos ejes independientes: el **estilo** (qué jugada le gusta más) y la **dificultad** (cuánto se equivoca).

Paso a paso (ComputerPlayer.chooseMove)

- **1. Lista de candidatas.** Recorre las piezas del bando que tiene el turno y le pide a la partida *game.legalMoves(origen)*. Así la IA nunca juega un movimiento ilegal ni deja a su rey en jaque: reutiliza las reglas del núcleo en vez de duplicarlas.
- **2. ¿Se equivoca?** Con la probabilidad de la dificultad elige una jugada al azar (Fácil 40 %, Media 15 %, Difícil 0 %).
- **3. Si no, la mejor.** Cada candidata pasa por *MoveAnalyzer*, que la **simula**: la hace con *game.move()*, mira cómo quedó el tablero y la revierte con *game.undo()*. De ahí salen los *MoveFacts*. El estilo convierte esos datos en un número y gana el mayor (los empates, al azar).

```
public Move chooseMove(IGame game) {
    List<Move> candidates = legalMoves(game);
    if (random.nextDouble() < difficulty.randomMoveProbability())
        return candidates.get(random.nextInt(candidates.size()));   // se "equivoca"
    return bestMove(game, candidates);                               // la de mayor puntaje
}
```

Datos que se calculan de cada jugada (MoveFacts)

Dato	Qué significa
capturedValue	Valor de la pieza que captura (0 si no captura).
givesCheck / givesCheckmate	Si deja al rey rival en jaque o en jaque mate.
hangingValue	Suma del valor de las piezas propias que el rival puede capturar después de la jugada.
kingShelter	Cuántas piezas propias quedan pegadas al rey propio.

¿Cuánto vale una pieza? *PieceValue* no tiene una tabla por tipo: el valor es la **movilidad** de la pieza en el centro de un tablero vacío del mismo tamaño (en 8x8: dama 27, torre 14, alfil 13, caballo 8, peón 1-2). Respeta el orden clásico y, si se agrega una pieza nueva o un tablero de otro tamaño, se valora solo.

Estilos de juego (patrón Strategy: IPlayStyle)

Estilo	Fórmula del puntaje	Comportamiento
Agresivo	capturado × 10 + 50 si da jaque	Captura y da jaque aunque deje piezas expuestas.
Defensivo	− colgado × 10 + refugio del rey × 5	Evita dejar piezas atacadas y protege al rey.
Equilibrado	Agresivo + Defensivo (composición)	Captura solo cuando es seguro.

Todos suman **100.000** si la jugada da jaque mate (CHECKMATE_BONUS), así que si hay mate, lo juegan. Agregar un estilo nuevo es escribir una clase con una fórmula: no se toca la IA ni el torneo.

Cómo se usa en el torneo

HistoricalPlayerFactory (patrón Factory) arma un plantel de **12 ajedrecistas históricos** (4 por dificultad), cada uno con un estilo y una dificultad. El torneo solo conoce la interfaz *IMoveChooser*, así que en los tests se reemplaza la IA por un rival guionado. El *Random* entra por constructor: con la misma semilla la IA juega siempre igual y los tests son deterministas.

2. ¿Cómo implementamos las piezas 3D?

El tablero 3D es la interfaz web, un **adaptador** más (adapters/web + src/main/resources/web). Se hizo con **Three.js** (WebGL) en JavaScript, incluido en *web/vendor* para que funcione sin internet. Un servidor **Javalin** en Java sirve la página y expone la partida como API HTTP/JSON.

Los modelos

Cada pieza es un archivo **.glb** (formato glTF binario, el estándar para 3D en la web) en *web/models*: **12 modelos**, 6 tipos × 2 colores (king, queen, rook, bishop, knight, pawn y sus versiones *_black*).

Cómo se cargan y se dibujan (pieces.js)

- **Carga única:** *loadModels()* usa *GLTFLoader* para bajar los 12 .glb una sola vez, en paralelo (Promise.all), y los guarda como plantillas. Una barra muestra el progreso.
- **Preparación:** a cada malla se le activan sombras y se ponen las texturas con filtro *Nearest* para que el pixel art se vea nítido y no borroso.
- **Clonado:** cada pieza del tablero es un clon de la plantilla (*SkeletonUtils.clone*) con sus materiales copiados, así se puede teñir el material "Base" de cada bando o hacerla brillar al seleccionarla sin afectar a las demás.
- **Detalles visuales:** escala 0,82 para dejar aire entre vecinas, un disco con degradé debajo como sombra de contacto, y las blancas giradas 180° para que los bandos se miren.
- **Escena (scene.js):** luz ambiente, luz principal con sombras suaves, contraluz y relleno; cámara en perspectiva con *OrbitControls* (girar arrastrando, zoom con la rueda, "Ver desde negras").
- **Interacción (input.js):** un *raycaster* detecta qué pieza o casilla está bajo el mouse; *anim.js* anima la pieza en arco al mover y el desvanecido al capturar.

Idea clave: el front no sabe ajedrez

El JavaScript **no tiene reglas**. Al hacer clic pide los destinos legales (*GET /api/game/moves?from=e2*), manda la jugada (*POST /api/game/move*) y el núcleo en Java decide. Después *PieceSet.sync(estado)* deja la escena igual al estado que devolvió el backend: saca las piezas que sobran, crea las que faltan y acomoda posiciones. La fuente de verdad es siempre el núcleo.

```
Clic en pieza  ->  GET /api/game/moves   ->  IGame.legalMoves()   (núcleo)
Clic en destino ->  POST /api/game/move  ->  IGame.move()         (núcleo)
Respuesta JSON  ->  PieceSet.sync(state) ->  la escena 3D coincide con el backend
```

Hay también una **vista 2D** (view2d.js) con la misma interfaz que la 3D: se cambia con un botón sin perder la partida, y si los .glb no cargan se juega igual en 2D.

3. ¿Por qué usamos base de datos?

Usamos **SQLite**: la base completa es un único archivo (*ajedrez.db*), sin servidor que instalar, y se crea sola la primera vez que se ejecuta el programa. Sin base, todo lo que pasa en el juego se pierde al cerrarlo; con ella hay **identidad** (quién juega) y **memoria** (qué jugó).

Tablas

Tabla	Columnas	Para qué
users	username (PK), password_hash	Las cuentas para iniciar sesión.
round_results	id, username (FK a users), rival, round, total_rounds, outcome, user_moves, played_at	Una fila por cada ronda de torneo terminada (ganada, perdida o tablas).

¿Por qué el inicio de sesión?

- El historial tiene que ser **de alguien**: sin cuenta no se puede saber de quién es cada partida.
- Varias personas pueden usar el mismo programa (o la misma web a la vez) y cada una ve **solo lo suyo**: en la web cada usuario tiene su propio tablero y su propio torneo.
- Seguridad: las contraseñas se guardan con **hash PBKDF2 + sal**, nunca en texto plano; en la web la sesión es una cookie *HttpOnly* con token aleatorio, y sin sesión la API responde 401.

¿Por qué el historial de partidas?

- Cada ronda queda guardada con rival, fecha, resultado y cantidad de jugadas.
- Con eso se calculan totales (ganadas, tablas, perdidas, títulos) y el récord del menú.
- Lo guarda *RecordingTournament* (patrón **Decorator**): envuelve al torneo y guarda cada ronda que termina, sin que la clase Tournament se entere de que existe una base.

Qué le aporta al usuario y qué al desarrollador

Para el USUARIO	Para el DESARROLLADOR
Tiene una **cuenta propia** y sus datos no se mezclan con los de otros.	Los datos **persisten** entre ejecuciones: no se pierden al cerrar el programa.
Ve su **progreso**: rondas jugadas, rivales, resultados, fechas y títulos.	Permite **autenticar** usuarios de forma segura (hash + sal) y separar los datos de cada uno.
Puede **volver otro día** y su historial sigue ahí.	La misma base sirve a los 3 adaptadores de entrada (consola, Swing y web): una sola fuente de datos.
Motivación: ve su récord (+ganadas =tablas −perdidas) y quiere mejorarlo.	Gracias a los **puertos de salida** el núcleo no conoce SQL: se puede cambiar SQLite por otra base escribiendo otro adaptador, y en los tests se usan repositorios **en memoria** (rápidos, sin archivo).
Su contraseña está protegida: ni el desarrollador la puede leer.	SQLite: cero instalación, un archivo, y se puede abrir para revisar datos y depurar. Consultas con PreparedStatement (sin inyección SQL).

4. Puertos, adaptadores y núcleo

El proyecto sigue la **arquitectura hexagonal**: un núcleo con las reglas, que no depende de nada externo, y adaptadores que se enchufan a él a través de interfaces llamadas puertos. *Main.java* es el único lugar donde se eligen las implementaciones concretas y se conectan.

1	6	4
núcleo	puertos (4 de entrada + 2 de salida)	adaptadores (3 de entrada + 1 de salida)

Núcleo: 1

El paquete **core**: **57 clases** en 10 subpaquetes (model, movement, setup, rules, commands, game, auth, history, ai, tournament). Tiene todas las reglas del ajedrez, la IA, el torneo, el login y el historial, con **cero dependencias externas** (ni SQL, ni Swing, ni Javalin).

Puertos: 6

Puerto	Tipo	Para qué	Quién lo usa / implementa
IGame	Entrada	Jugar una partida libre: mover, jugadas legales, deshacer/rehacer, estado.	Consola, Swing, Web
ITournament	Entrada	Jugar el torneo escalera contra la IA.	Consola, Swing, Web
IAuthService	Entrada	Crear cuenta e iniciar sesión.	Consola, Swing, Web
IHistoryService	Entrada	Consultar el historial y los totales del usuario.	Consola, Swing, Web
IUserRepository	Salida	Guardar y buscar cuentas.	JdbcUserRepository (SQLite)
IMatchHistoryRepository	Salida	Guardar y leer las rondas jugadas.	JdbcMatchHistoryRepository (SQLite)

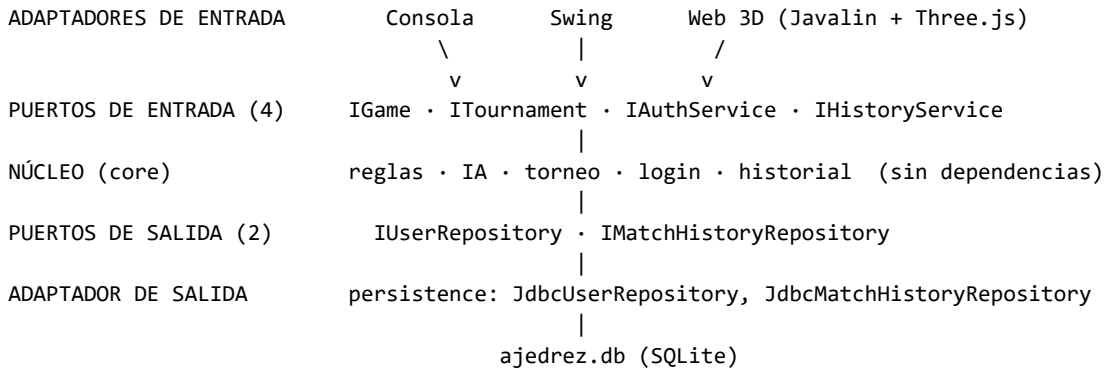
Entrada = por donde el mundo de afuera le pide cosas al núcleo. **Salida** = lo que el núcleo necesita del mundo de afuera (guardar datos); el núcleo define la interfaz y el adaptador la implementa.

Adaptadores: 4

Adaptador	Tipo	Clases	Qué hace
console	Entrada	6	Juego por texto: ConsoleAdapter, TournamentConsoleAdapter, ConsoleLogin, HistoryPrinter, ...
gui	Entrada	18	Interfaz gráfica Swing: login, menú, partida, torneo, historial.
web	Entrada	9	Servidor Javalin (API HTTP/JSON) + front Three.js con el tablero 3D.
persistence	Salida	4	SQLite: SqliteDatabase, JdbcUserRepository, JdbcMatchHistoryRepository, DatabaseException.

Los tres adaptadores de entrada usan **los mismos 4 puertos**: por eso el mismo juego funciona en consola, en ventana y en el navegador sin tocar el núcleo. En los tests, los puertos de salida se implementan con repositorios en memoria (InMemoryUserRepository, InMemoryMatchHistoryRepository).

Diagrama



Aclaraciones para la defensa

- **IPasswordHasher**, **IPlayStyle**, **IMoveChooser**, **IMovementRule**, etc. también son interfaces, pero son internas del núcleo (patrón Strategy), no puertos: no las usa ni las implementa ningún adaptador.
- No confundir con el **puerto de red 7070** donde escucha el servidor web (http://localhost:7070); eso es otra cosa, de redes, no de la arquitectura.
- **Main.java** no es núcleo ni adaptador: es la raíz de composición, donde se inyectan las dependencias.